



CLOUD DATA ENGINEERING · END-TO-END CASE STUDY

InsightStream

Cloud Data Warehouse & Analytics

Architecture · Testing · Documentation · Results

Team Leader

Youssef Ahmed Mohamed Alkamashany

Team Members

Abdelrahman Adel Abdelmola Abu Taleb • Abdullah Mohamed Ahmed • Omar Abdelgawad Mahmoud

A Complete, Reproducible Analytics Platform

Raw, messy e-commerce order data from a Brazilian marketplace is ingested, staged, cleaned, modeled, tested, orchestrated daily, documented at three independent levels, and surfaced as a live BI dashboard — every step reproduced and explained.

The real engineering value isn't the dashboard — it's everything invisible built correctly first: a layered warehouse architecture, a four-level testing strategy that caught three real infrastructure defects, and documentation deep enough to rebuild the system from a blank machine.

≈100K

Real customer orders processed

4

Warehouse layers: raw → silver
→ gold → BI

3

Real infrastructure defects found
& fixed

24/7

Daily automated orchestration

What You'll Find Inside



01 • System Architecture

The ELT design, medallion data model, and the real-world dataset that powers it all.



02 • Testing

A four-level testing strategy, three real defects, and proof of reproducible runs.



03 • Documentation

Three independent layers designed so the system outlives its author.



04 • Results

A live Power BI dashboard built on governed data — and what the numbers say.



05 • Conclusion & Roadmap

What InsightStream proves, and the concrete next steps.

Why ELT Instead of ETL?

InsightStream implements a modern ELT pipeline built entirely on cloud-native, industry-standard tools. Raw data is loaded first and transformed afterward inside the warehouse — taking advantage of Snowflake's elastic compute to run transformations at scale using SQL.

Traditional ETL

- Transformation logic lives in a separate processing engine before data reaches the warehouse.
- Every schema change requires touching pipeline code outside the warehouse.
- A single ETL server can bottleneck at scale.

ELT (InsightStream)

- Raw data lands first, untouched; every transformation is a version-controlled SQL model.
- Snowflake's compute scales independently of storage — transformations run as parallel, elastic SQL jobs.
- Raw source-of-truth stays in Bronze, ready to be re-run through corrected logic anytime.

End-to-End ELT Architecture

Orchestrated end-to-end by Apache Airflow — DAG: load_raw_data → dbt_run → dbt_test

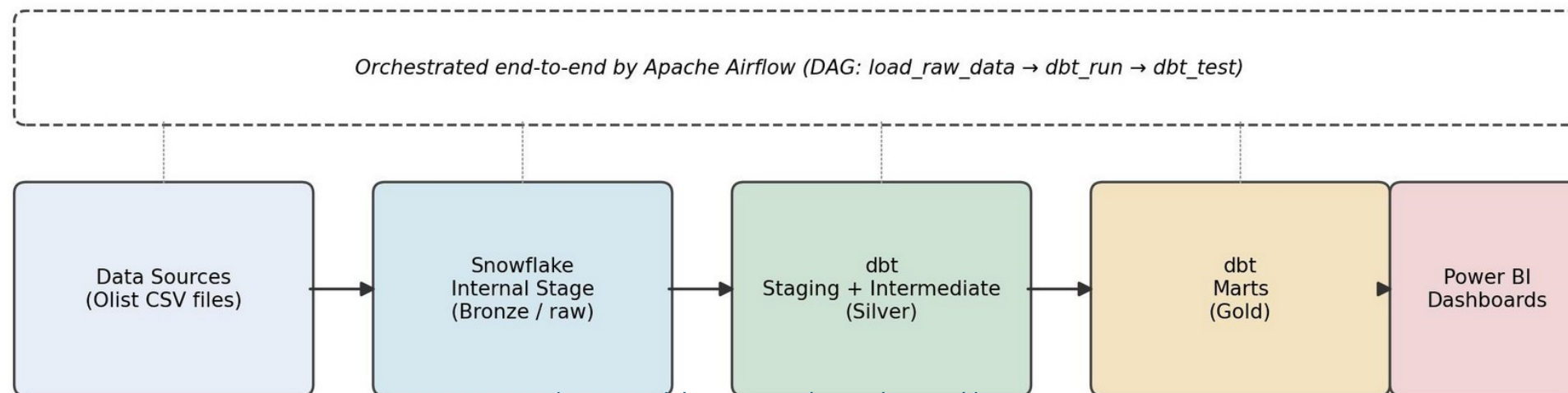


Figure 1: InsightStream End-to-End ELT Architecture

Figure 1: InsightStream ELT Architecture

Bronze never lies — an exact, immutable copy of what arrived. Silver is where messy reality gets cleaned up: types fixed, columns renamed, entities joined. Gold is the only layer a business user should ever query directly, since every quality check has already been applied.

Component Breakdown

Layer	Technology	Purpose
Ingestion	Python + snowflake-connector-python	Reads local CSV source files and loads them into Snowflake
Bronze (raw)	Snowflake Internal Stage + raw schema	Stores untransformed source data exactly as ingested
Silver	dbt staging & intermediate models	Cleans column types, renames fields, joins related entities
Gold	dbt marts (fact / dimension models)	fct_orders, dim_customers, dim_products, mart_sales_summary
Orchestration	Apache Airflow (Dockerized, LocalExecutor)	Schedules ingestion → dbt run → dbt test daily
Presentation	Power BI Desktop	Interactive dashboards connected directly to the Gold layer

Data Model — dbt Layers

Staging

One model per source table. Only type casting and column renaming — no business logic.

Intermediate

int_orders_enriched joins orders, order items, products, customers, and aggregated payments into a single denormalized view.

Marts

dim_customers & dim_products (dimensions), fct_orders (fact), mart_sales_summary (pre-aggregated by state, category, month).



Engineering Insight — Separation of Concerns

Staging models do exactly one thing, so they stay trivially easy to audit. Business logic lives only in intermediate and marts. If a dashboard number looks wrong, an engineer walks backward through exactly three well-defined hops — mart → intermediate → staging → raw — instead of untangling one monolithic script.

Orchestration Design

The Airflow DAG (insightstream_elt_pipeline) is intentionally linear with three sequential tasks. Each task only proceeds once the previous one succeeds — guaranteeing transformations never run against incomplete or stale raw data.



A linear DAG is trivial to reason about during a 2 a.m. on-call page — there is exactly one possible point of failure to check at a time. As the pipeline grows, it can be incrementally parallelized (e.g. running independent dbt model groups concurrently) without ever losing the simple, auditable backbone of load → transform → test.

Data Source — Real, Not Synthetic

The pipeline is fed by the Brazilian E-Commerce Public Dataset by Olist (Kaggle): close to 100,000 real orders placed between 2016–2018 across customers, products, sellers, payments and reviews — split across 8 related CSV files.

- Product category names recorded only in Portuguese
- Payments split across multiple installments and methods per order
- Geolocation data that must survive every join without dropping rows

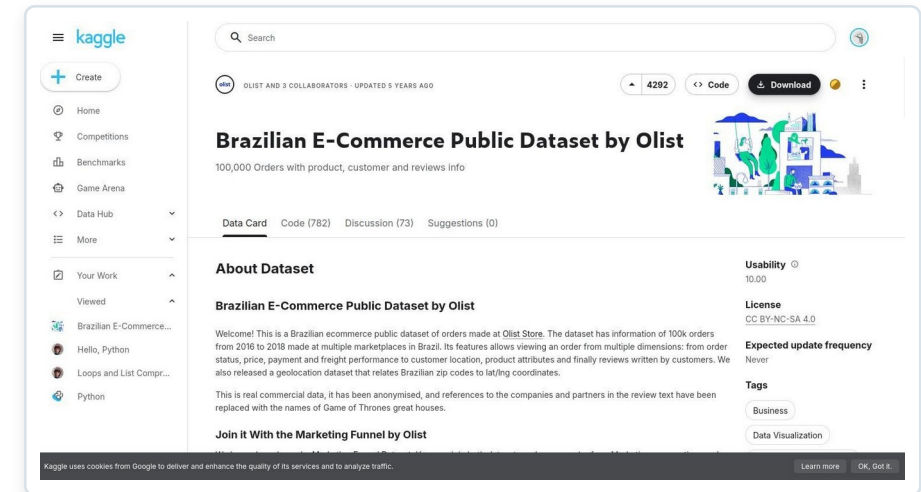
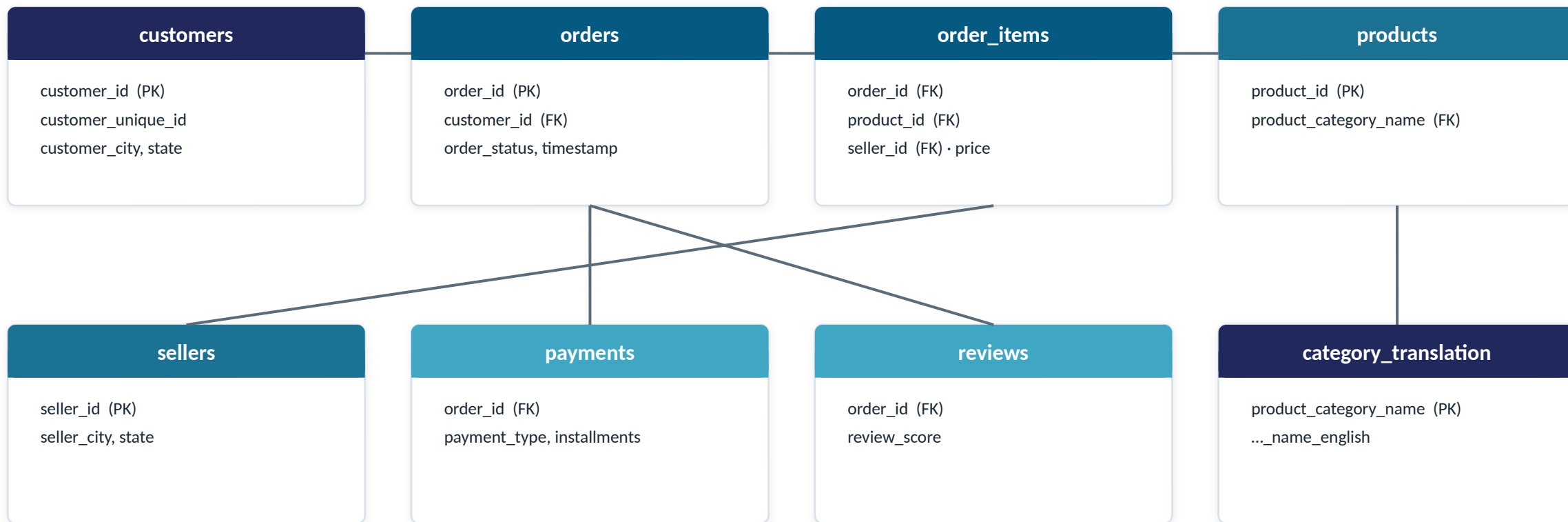


Figure 2: Source Dataset — Brazilian E-Commerce Public Dataset by Olist (Kaggle)

Why real data matters: synthetic data is clean by construction — real data offers no such guarantees, which is exactly why it's a more valuable, more realistic testing ground.

Data Modeling — How the Tables Connect

The 8 raw source tables are relational, not standalone — each carries keys that link it to at least one other table. Mapping these relationships correctly is what makes the staging → intermediate → marts pipeline meaningful.



This is the same relational backbone shown in the dbt lineage graph (Section 3) — every Gold-layer number can be traced back through these keys to a specific raw record.

A Four-Level Testing Strategy

Testing moves from the smallest unit of logic up to the full pipeline running end-to-end — the classic “testing pyramid” applied to a data pipeline.



Testing Levels & Data Quality Checks

Type	What Was Tested	Tool
Unit	Python ingestion functions (table creation, file upload, COPY INTO) verified in isolation	Manual execution in Airflow container
Data Quality	Uniqueness/not-null on primary keys; referential integrity between fct_orders & dim_customers	dbt test (schema.yml)
Integration	Full flow: source CSVs → Snowflake raw → dbt transforms → queryable marts	Airflow DAG run
Performance	dbt model build time and Power BI dashboard query response time	dbt run timing / Power BI



Key checks: unique + not_null on customer_id and product_id • not_null on order_id • not_null + relationships on fct_orders.customer_id → dim_customers

Three Real Defects — Found & Fixed

Issue	Root Cause	Resolution
Airflow task logs returned HTTP 403 and were unreadable	Webserver and scheduler containers each generated a different random Flask secret key	Set a fixed AIRFLOW__WEBSERVER__SECRET_KEY identically across all Airflow services
Snowflake error 090105: no current database for CREATE TABLE	The Python connector session did not retain the database/schema context	Added explicit role plus USE WAREHOUSE / DATABASE / SCHEMA statements right after connecting
Snowflake error 002043: object does not exist	Warehouse/database objects had not been created successfully beforehand	Re-ran setup with fully qualified object names and verified with SHOW WAREHOUSES



Worth highlighting, not hiding: none of these were SQL logic bugs — every one was an infrastructure/config issue that only became visible once components were forced to talk to each other in a real Docker/Airflow/Snowflake environment. That's exactly what integration testing exists to catch.

Execution Evidence & Reproducibility

The insightstream_elt_pipeline DAG is active, scheduled to run @daily, with a history of completed runs and the next run already queued — confirming the orchestration layer is live, not a one-off manual script.

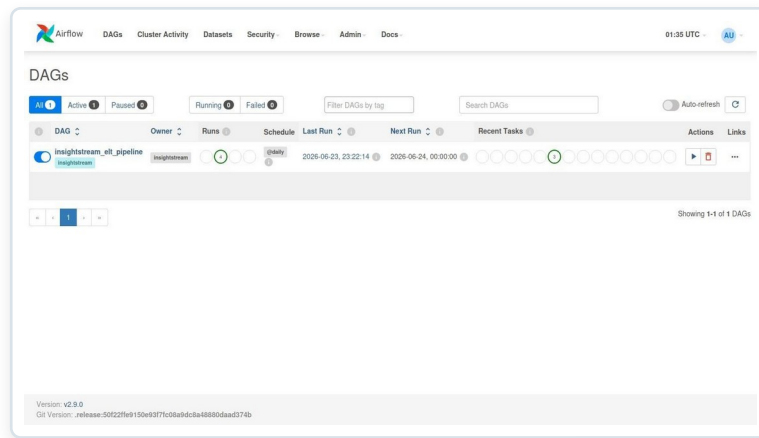


Figure 3: DAG Registered with a Daily Schedule

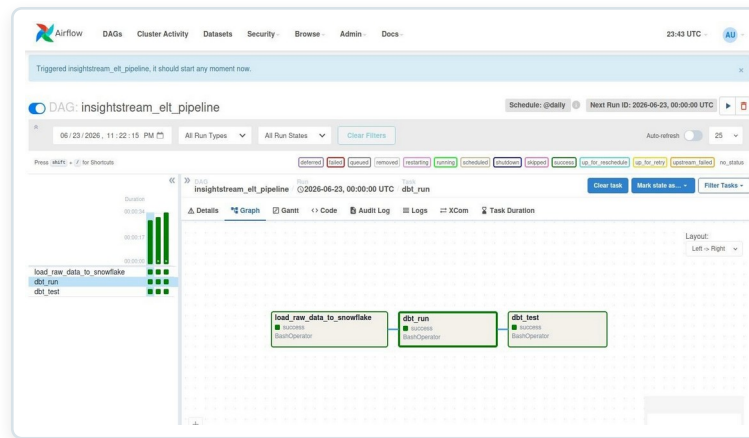


Figure 4: Successful Pipeline Run — Attempt 1

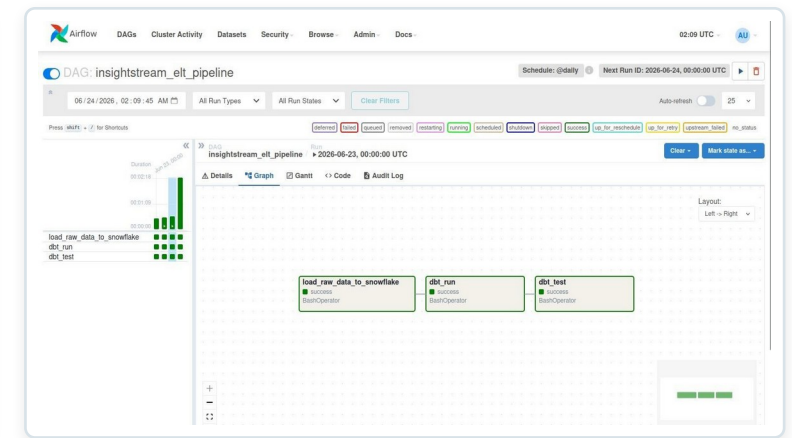


Figure 5: Successful Run — Attempt 2 (Reproducibility)

Reproducibility is an underrated quality signal — a pipeline that succeeds twice, independently, on different days is starting to demonstrate the reliability a production schedule demands.

Three Layers That Outlive the Author

Documentation was produced at three levels — auto-generated technical docs from dbt, human-written setup/run instructions, and inline code documentation — so a new team member could operate the system without the original author.

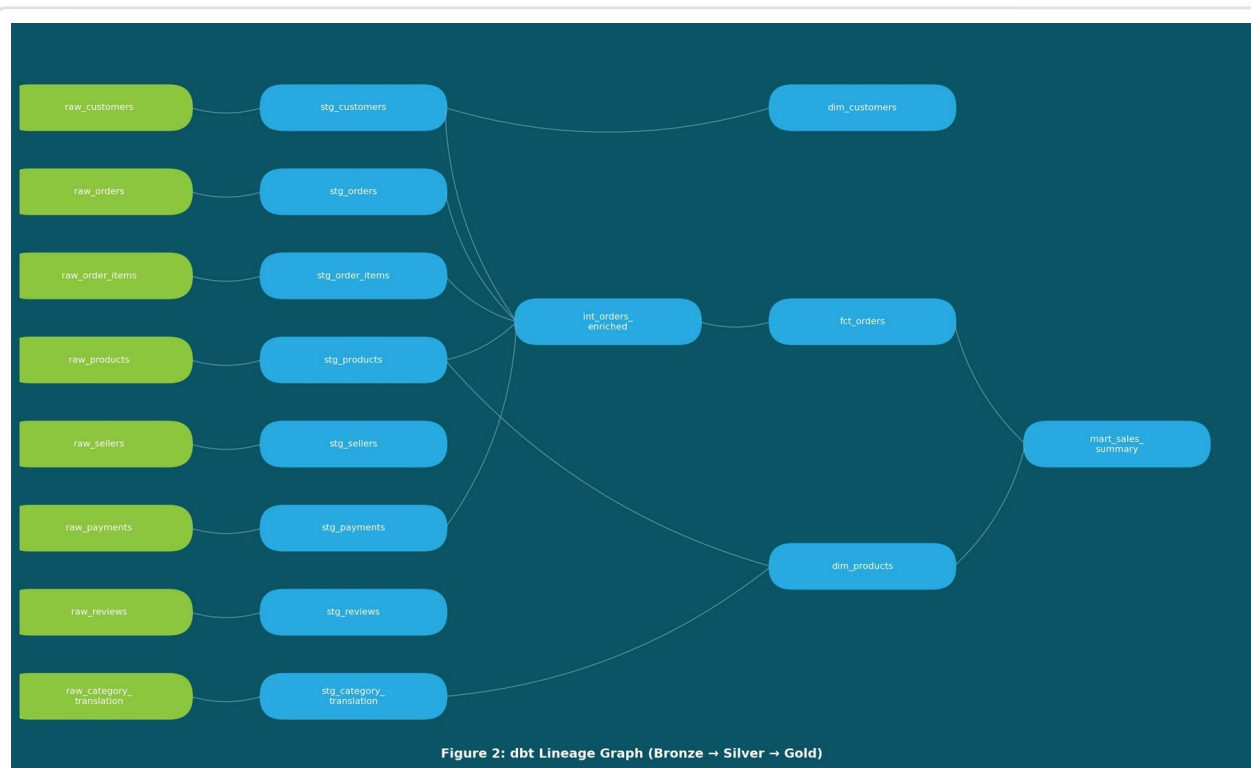


Figure 6: dbt Lineage Graph (Bronze → Silver → Gold)



README (Setup)

Environment prerequisites, Snowflake setup, credentials, Docker stack, DAG triggering, and table verification.



Inline Code Docs

Docstrings and single-purpose ingestion functions; dbt models organized into staging/intermediate/marts; DAG dependencies via >>.

Bus factor: redundant docs are insurance, not waste.

Business Intelligence Dashboard

Gold-layer tables (fct_orders, dim_products, mart_sales_summary) are consumed directly by Power BI Desktop over a live Snowflake connection.



Figure 7: InsightStream Sales Dashboard (Power BI)

Total recorded sales: 100,245 units of value, aggregated entirely through the dbt-modelled Gold layer.

Key Observations



Seasonality signal

Monthly sales peak around May, dip around September, and recover by November — a clear planning signal for inventory and staffing.



Geography validated

Order volume concentrates in Brazil, consistent with the dataset's origin — confirming location data survived every join.



Category breakdown

Product category totals support fast inventory and marketing prioritization decisions.



Governed numbers

All totals are aggregated entirely through the dbt-modelled Gold layer, not raw, unprocessed data.

What InsightStream Demonstrates

A small, carefully layered pipeline can deliver the same core guarantees as a much larger enterprise data platform: clean separation between raw and modeled data, automated data-quality enforcement, daily unattended orchestration, multi-level documentation, and a governed, trustworthy path through to a business-facing dashboard.

Where InsightStream Goes Next

- Incremental loading — dbt incremental models + Snowflake Streams/Tasks
- CI/CD — GitHub Actions running dbt build & dbt test on every pull request
- Alerting — Airflow task failures routed to Slack or email
- Deeper BI — DAX measures (MoM growth, CLV) and row-level security
- Forecasting — extend the seasonality pattern into a demand-forecasting model

“Build the foundation so solidly that the dashboard becomes the easy part.”